UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

CARRERA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS

GUÍA DE LABORATORIO N.º 06

Algoritmos fundamentales de teoría de números

Asignatura:	Algoritmos Avanzados
Unidad:	II. Estructuras de datos avanzadas y fundamentos algebraicos
Semestre académico:	2026-II
Duración:	120 minutos
Organización:	Trabajo grupal en los equipos del proyecto semestral
Integrantes:	Aproximadamente cuatro estudiantes por equipo
Lenguaje:	Python 3
Docente:	Héctor Eduardo Ugarte Rojas

Equipo: _____

Integrantes: _____

Fecha: _____

Índice

1. Objetivo de la práctica **2**

2. Trabajo preparatorio obligatorio **2**

3. Materiales **2**

4. Marco teórico breve **3**

4.1. Algoritmo de Euclides . . . 3

4.2. Euclides extendido e inversos . . . 3

4.3. Exponenciación modular binaria . . . 3

4.4. Congruencias lineales y TCR . . . 3

5. Especificación mínima de la implementación **3**

6. Propiedades que deben comprobarse **4**

7. Distribución sugerida de los 120 minutos **4**

8. Ejercicios resueltos **4**

8.1. Ejercicio resuelto 1: Euclides con traza y coeficientes de Bézout . . . 4

8.2. Ejercicio resuelto 2: inversos y congruencias lineales . . . 5

8.3. Ejercicio resuelto 3: exponenciación modular binaria instrumentada . . . 6

9. Ejercicios propuestos **7**

9.1. Ejercicio propuesto 1: biblioteca integrada y teorema chino del resto . . . 7

9.2. Ejercicio propuesto 2: pruebas diferenciales y análisis experimental . . . 8

10. Metodología de medición **8**

11. Resultados mínimos requeridos **9**

12. Preguntas de análisis **9**

13. Producto de entrega **10**

14. Tabla de evaluación **10**

15. Casos mínimos de prueba **11**

16. Referencias **11**

1. Objetivo de la práctica

Objetivo

Implementar y validar algoritmos fundamentales de teoría de números: máximo común divisor, algoritmo extendido de Euclides, inverso modular, congruencias lineales, exponenciación modular rápida y teorema chino del resto. Posteriormente, comprobar sus propiedades mediante pruebas automatizadas y analizar experimentalmente el ahorro de operaciones obtenido por la exponenciación binaria.

Al finalizar la práctica, el equipo será capaz de:

1. implementar el algoritmo de Euclides y registrar su traza;
2. obtener coeficientes de Bézout mediante Euclides extendido;
3. calcular inversos modulares y detectar cuándo no existen;
4. resolver congruencias lineales y enumerar todas sus soluciones;
5. implementar exponenciación modular binaria sin construir la potencia completa;
6. resolver sistemas compatibles mediante el teorema chino del resto;
7. contrastar los resultados con funciones incorporadas de Python y verificadores directos;
8. analizar el número de operaciones y el efecto del tamaño en bits de las entradas.

2. Trabajo preparatorio obligatorio

Cada integrante deberá revisar:

1. división euclidiana y significado del cociente y el residuo;
2. definición de $\mathrm{mcd}(a, b)$ y condición de terminación de Euclides;
3. identidad de Bézout $ax + by = \mathrm{mcd}(a, b)$;
4. definición de congruencia y clase residual;
5. existencia del inverso modular: $\mathrm{mcd}(a, n) = 1$;
6. representación binaria de un exponente;
7. condición de solubilidad de $ax \equiv b \pmod{n}$;
8. hipótesis del teorema chino del resto.

3. Materiales

- Computadora con Python 3.11 o posterior, recomendado.
- Editor o IDE, terminal y Git.
- Biblioteca estándar: `math`, `random`, `time`, `statistics`, `csv` y `secrets`.
- Repositorio del equipo con la estructura solicitada.
- Diapositivas del tema 6 y casos de prueba preparados por el equipo.
- Herramienta para generar tablas y gráficos a partir del archivo CSV.

4. Marco teórico breve

4.1. Algoritmo de Euclides

Si $a = bq + r$, entonces:

$$\mathrm{mcd}(a, b) = \mathrm{mcd}(b, r).$$

El algoritmo reemplaza repetidamente (a, b) por $(b, a \bmod b)$ hasta que el segundo componente sea cero. El último residuo no nulo es el MCD. La sucesión de residuos disminuye estrictamente, por lo que el algoritmo termina.

4.2. Euclides extendido e inversos

La versión extendida calcula d, x, y tales que:

$$d = \mathrm{mcd}(a, b), \qquad ax + by = d.$$

El inverso de a módulo n existe si y solo si $\mathrm{mcd}(a, n) = 1$. En ese caso, si $ax + ny = 1$, entonces:

$$a^{-1} \equiv x \pmod{n}.$$

4.3. Exponenciación modular binaria

La representación binaria del exponente permite calcular $a^e \bmod n$ mediante cuadrados sucesivos. La potencia directa necesita $\Theta(e)$ multiplicaciones; el método binario necesita $\mathcal{O}(\log e)$ multiplicaciones modulares y puede usar $\mathcal{O}(1)$ memoria auxiliar.

4.4. Congruencias lineales y TCR

La congruencia:

$$ax \equiv b \pmod{n}$$

tiene solución si y solo si $d = \mathrm{mcd}(a, n)$ divide a b. Cuando existe, tiene exactamente d soluciones distintas módulo n.

Si $n_1, \ldots, n_k$ son coprimos dos a dos, el sistema:

$$x \equiv a_i \pmod{n_i}$$

tiene una única solución módulo $N = \prod_i n_i$. Definiendo $N_i = N/n_i$ y $M_i = N_i^{-1} \bmod n_i$:

$$x \equiv \sum_i a_i N_i M_i \pmod{N}.$$

5. Especificación mínima de la implementación

Listing 1: Interfaz requerida

```python
def gcd_trace(a, b): ...
def extended_gcd(a, b): ...
def modular_inverse(a, n): ...
def solve_linear_congruence(a, b, n): ...
def modular_pow(base, exponent, modulus): ...
def crt(residues, moduli): ...
def validate_crt_solution(x, residues, moduli): ...
```

Convenciones de resultados

`gcd_trace` devolverá el MCD y la lista de divisiones realizadas. `extended_gcd` devolverá (d, x, y). `modular_inverse` deberá rechazar los casos sin inverso. `solve_linear_congruence` devolverá todas las soluciones normalizadas en $\{0, \ldots, n-1\}$. `crt` devolverá (x, N), donde x es el menor representante no negativo.

6. Propiedades que deben comprobarse

Después de cada operación de prueba, el equipo deberá verificar:

1. **Traza válida:** cada fila satisface $a_i = b_i q_i + r_i$ y $0 \leq r_i < |b_i|$.
2. **MCD correcto:** el resultado coincide con `math.gcd`.
3. **Bézout:** la salida extendida cumple $ax + by = d$.
4. **Inverso:** si existe $u = a^{-1}$ mód n, entonces (au) mód $n = 1$.
5. **No existencia:** si $\text{mcd}(a, n) \neq 1$, el algoritmo no devuelve un inverso falso.
6. **Congruencia lineal:** cada solución cumple $(ax - b)$ mód $n = 0$.
7. **Cantidad de soluciones:** si $d \mid b$, se obtienen exactamente d soluciones módulo n.
8. **Potencia modular:** el resultado coincide con `pow(a,e,n)`.
9. **TCR:** la solución satisface simultáneamente todas las congruencias.
10. **Normalización:** los residuos retornados pertenecen al intervalo solicitado.

7. Distribución sugerida de los 120 minutos

Etapa	Actividad	Tiempo
Inicio	Revisión de interfaces, convenciones y propiedades	10 min
Resuelto 1	Euclides con traza y Euclides extendido	20 min
Resuelto 2	Inversos y congruencias lineales	20 min
Resuelto 3	Exponenciación binaria y conteo de operaciones	18 min
Propuesto 1	Biblioteca integrada y teorema chino del resto	30 min
Propuesto 2	Pruebas diferenciales y análisis experimental	17 min
Cierre	Verificación individual y organización de evidencias	5 min

8. Ejercicios resueltos

8.1. Ejercicio resuelto 1: Euclides con traza y coeficientes de Bézout

El siguiente programa registra las divisiones del algoritmo y calcula los coeficientes de Bézout mediante la forma iterativa de Euclides extendido.

Listing 2: MCD, traza y algoritmo extendido

```
def gcd_trace(a, b):
    a, b = abs(a), abs(b)
```

```
    trace = []

    while b != 0:
        q, r = divmod(a, b)
        trace.append((a, b, q, r))
        a, b = b, r

    return a, trace


def extended_gcd(a, b):
    old_r, r = abs(a), abs(b)
    old_s, s = 1, 0
    old_t, t = 0, 1

    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
        old_t, t = t, old_t - q * t

    x = old_s if a >= 0 else -old_s
    y = old_t if b >= 0 else -old_t
    return old_r, x, y


d, trace = gcd_trace(252, 105)
assert d == 21
assert trace == [
    (252, 105, 2, 42),
    (105, 42, 2, 21),
    (42, 21, 2, 0),
]

d, x, y = extended_gcd(252, 105)
assert (d, x, y) == (21, -2, 5)
assert 252 * x + 105 * y == d
```

Interpretación. La traza explica la terminación de Euclides. La versión extendida mantiene cada residuo como combinación lineal de los números originales. Para el ejemplo:

$$21 = 252(-2) + 105(5).$$

8.2. Ejercicio resuelto 2: inversos y congruencias lineales

Listing 3: Inverso modular y soluciones de una congruencia

```
def modular_inverse(a, n):
    if n <= 1:
        raise ValueError("El modulo debe ser mayor que uno")

    d, x, _ = extended_gcd(a, n)
    if d != 1:
        raise ValueError("El inverso modular no existe")
    return x % n


def solve_linear_congruence(a, b, n):
    if n <= 0:
        raise ValueError("El modulo debe ser positivo")
```

```
    d, x, _ = extended_gcd(a, n)
    if b % d != 0:
        return []

    reduced_modulus = n // d
    x0 = (x * (b // d)) % reduced_modulus
    return [
        (x0 + k * reduced_modulus) % n
        for k in range(d)
    ]


assert modular_inverse(7, 40) == 23
assert (7 * 23) % 40 == 1

solutions = solve_linear_congruence(14, 30, 100)
assert solutions == [45, 95]
assert all((14 * x - 30) % 100 == 0 for x in solutions)
```

En $14x \equiv 30 \pmod{100}$, $d = \gcd(14, 100) = 2$ y $2 \mid 30$. Por ello existen dos soluciones distintas módulo 100:

$$\boxed{x \equiv 45, 95 \pmod{100}}.$$

8.3. Ejercicio resuelto 3: exponenciación modular binaria instrumentada

El programa contabiliza cuadrados y multiplicaciones aplicadas al resultado. Esto permite comparar el método binario con la multiplicación repetida.

Listing 4: Exponenciación modular y conteo de operaciones

```
def modular_pow(base, exponent, modulus):
    if modulus <= 0:
        raise ValueError("El modulo debe ser positivo")
    if exponent < 0:
        raise ValueError("El exponente debe ser no negativo")

    result = 1 % modulus
    base %= modulus
    squares = 0
    selected_products = 0

    while exponent > 0:
        if exponent & 1:
            result = (result * base) % modulus
            selected_products += 1

        exponent >>= 1
        if exponent > 0:
            base = (base * base) % modulus
            squares += 1

    return result, squares, selected_products


value, squares, products = modular_pow(7, 13, 20)
assert value == pow(7, 13, 20) == 7
assert squares == 3
assert products == 3
```

Para $13 = (1101)_2$, se generan sucesivamente a^2, a^4 y a^8 mediante tres cuadrados. Se seleccionan a, a^4 y a^8 porque los bits correspondientes son 1. En una implementación acumulativa se realizan tres multiplicaciones sobre el resultado, aunque la primera sea $1 \cdot a$. El número total permanece en $\mathcal{O}(\log e)$.

9. Ejercicios propuestos

9.1. Ejercicio propuesto 1: biblioteca integrada y teorema chino del resto

Complete una biblioteca denominada `number_theory.py` que reúna las funciones de la interfaz requerida e implemente el teorema chino del resto para módulos coprimos dos a dos.

Requisitos funcionales:

1. validar que residuos y módulos tengan la misma longitud;
2. rechazar módulos menores o iguales que uno;
3. comprobar que los módulos sean coprimos dos a dos;
4. calcular N, cada $N_i = N/n_i$ y cada $M_i = N_i^{-1}$ mód n_i;
5. devolver el menor representante no negativo x y el módulo total N;
6. generar una traza con las columnas $i, a_i, n_i, N_i, M_i, a_i N_i M_i$;
7. incluir un verificador que compruebe todas las congruencias;
8. evitar búsquedas exhaustivas de la solución;
9. documentar precondiciones, poscondiciones y excepciones.

Caso obligatorio:

$$\begin{cases} x \equiv 2 \pmod 3, \\ x \equiv 3 \pmod 5, \\ x \equiv 2 \pmod 7. \end{cases}$$

La salida debe ser:

$$(x, N) = (23, 105).$$

La traza deberá mostrar:

$$(N_1, M_1) = (35, 2), \quad (N_2, M_2) = (21, 1), \quad (N_3, M_3) = (15, 1).$$

Pruebas obligatorias:

1. un sistema de una sola congruencia;
2. el caso clásico anterior;
3. residuos negativos y mayores que el módulo;
4. módulos en distinto orden;
5. listas vacías o de diferente longitud;
6. módulos repetidos o no coprimos;
7. comparación con búsqueda exhaustiva exclusivamente para productos pequeños;
8. sistemas aleatorios construidos a partir de una solución conocida.

9.2. Ejercicio propuesto 2: pruebas diferenciales y análisis experimental

Diseñe experimentos reproducibles para evaluar corrección y número de operaciones.

Escenario A: pruebas diferenciales Para al menos 5000 pares aleatorios (a, b) con longitudes de 8, 16, 32, 64, 128, 256 y 512 bits:

1. compare `gcd_trace` con `math.gcd`;
2. compruebe la identidad de Bézout para `extended_gcd`;
3. registre la cantidad de divisiones realizadas;
4. conserve la semilla y la entrada en caso de discrepancia.

Escenario B: potencia modular Para exponentes con 8, 16, 32, 64, 128, 256, 512 y 1024 bits:

1. compare `modular_pow` con `pow(base, exponent, modulus)`;
2. registre cuadrados, productos seleccionados, tiempo y longitud binaria del exponente;
3. compare el conteo con las $e - 1$ multiplicaciones del método directo, sin ejecutar el método directo cuando e sea excesivo;
4. interprete cómo el número de bits y la cantidad de bits iguales a 1 afectan el trabajo.

Escenario C: inversos y congruencias Genere casos con y sin solución, verificando que:

- el inverso exista exactamente cuando el MCD sea 1;
- cada solución de $ax \equiv b \pmod{n}$ sea válida;
- la cantidad de soluciones sea la prevista por el MCD;
- el sistema TCR devuelva una solución equivalente a la solución conocida usada para generar los residuos.

Hipótesis experimental

El propósito del experimento no es demostrar las cotas asintóticas únicamente con tiempos. El informe debe distinguir cantidad de iteraciones, costo de operaciones sobre enteros grandes, comportamiento teórico y rendimiento observado en Python.

10. Metodología de medición

Listing 5: Medición básica y reproducible

```python
from statistics import median
from time import perf_counter_ns


def measure(function, repetitions=10):
    times = []
    for _ in range(repetitions):
        start = perf_counter_ns()
        function()
        end = perf_counter_ns()
        times.append(end - start)
```

```
    return {
        "min_ns": min(times),
        "median_ns": median(times),
        "max_ns": max(times),
    }
```

Para disminuir ruido experimental:

- genere las entradas antes de iniciar la medición;
- use las mismas entradas para los algoritmos comparados;
- verifique primero la corrección;
- utilice semillas fijas para las pruebas reproducibles;
- reporte mediana, mínimo y máximo de varias ejecuciones;
- no incluya impresiones ni escritura de archivos dentro del intervalo medido;
- registre la longitud en bits con `int.bit_length()`.

11. Resultados mínimos requeridos

Los resultados mínimos son:

1. tabla con MCD, cantidad de divisiones y longitud en bits;
2. comprobación automática de Bézout para todos los casos;
3. tabla de inversos existentes y casos rechazados;
4. tabla de congruencias lineales con cantidad de soluciones esperada y obtenida;
5. traza completa del caso obligatorio del TCR;
6. gráfico de divisiones de Euclides frente a longitud en bits;
7. gráfico de operaciones de potencia modular frente a bits del exponente;
8. comparación entre $e - 1$ y el conteo del método binario;
9. registro de casos, semillas y resultados en CSV.

12. Preguntas de análisis

Responda utilizando evidencia de la implementación y los experimentos:

1. ¿Por qué la sucesión de residuos de Euclides necesariamente termina en cero?
2. ¿Qué información adicional conserva Euclides extendido?
3. ¿Por qué los coeficientes de Bézout no son únicos?
4. ¿Por qué a tiene inverso módulo n exactamente cuando $\gcd(a, n) = 1$?
5. ¿Qué diferencia existe entre a^{-1} como fracción y como inverso modular?

6. ¿Por qué no siempre puede cancelarse un factor en una congruencia?
7. ¿Cómo determina el MCD la existencia y cantidad de soluciones de una congruencia lineal?
8. ¿Qué ahorro produce la descomposición binaria del exponente?
9. ¿Cómo influye la cantidad de bits iguales a 1 en las multiplicaciones del resultado?
10. ¿Por qué la reducción modular temprana evita números intermedios enormes?
11. ¿Qué función cumplen N_i y M_i en el TCR?
12. ¿Por qué la fórmula directa del TCR requiere módulos coprimos dos a dos?

13. Producto de entrega

Producto de entrega

El equipo entregará:

1. código fuente completo y organizado, incluyendo ejercicios resueltos y propuestos;
2. pruebas automatizadas de propiedades y casos excepcionales;
3. biblioteca `number_theory.py` documentada;
4. generador reproducible de casos aleatorios;
5. tablas y gráficos solicitados;
6. informe PDF con todo lo anterior y metodología, resultados y respuestas de análisis;
7. conclusiones grupales de 250 a 350 palabras.

14. Tabla de evaluación

Criterio	Indicador de logro	Puntaje
Asistencia y participación en el laboratorio	Asiste, permanece durante la práctica y participa en implementación, pruebas, análisis y revisiones. Se determina individualmente.	4
Corrección y funcionamiento	Los algoritmos de Euclides, Bézout, inversos, congruencias, potencia modular y TCR funcionan; las pruebas verifican sus propiedades y los resultados coinciden con las referencias utilizadas.	8
Análisis de complejidad y resultados	Relaciona divisiones, tamaño en bits, cuadrados, productos seleccionados y costo temporal; interpreta los experimentos sin confundir iteraciones con tiempo constante.	3
Dominio individual	Explica los algoritmos, ejecuta una traza, verifica una identidad o congruencia y modifica o ejecuta una prueba durante la revisión.	4
Total		**19**

15. Casos mínimos de prueba

- $\gcd(a, 0)$, $\gcd(0, b)$ y rechazo o tratamiento documentado de $(0, 0)$.
- Entradas positivas, negativas e intercambiadas.
- Números coprimos y no coprimos.
- Pares consecutivos de Fibonacci.
- Inverso existente, inverso inexistente y residuo negativo.
- Congruencia lineal sin solución, con una solución y con varias soluciones.
- Exponente cero, base cero, módulo uno y exponentes con muchos bits.
- Exponentes binarios con pocos y muchos bits iguales a 1.
- TCR con una congruencia, tres congruencias y residuos no normalizados.
- TCR con módulos repetidos o no coprimos.
- Pruebas aleatorias comparadas con `math.gcd` y `pow`.

16. Referencias

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022.

[2] V. Shoup. *A Computational Introduction to Number Theory and Algebra*, 2.ª ed. Cambridge University Press, 2009.

[3] D. E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3.ª ed. Addison-Wesley, 1997.

[4] Python Software Foundation. *Mathematical functions*. https://docs.python.org/3/library/math.html

[5] Python Software Foundation. *Built-in Functions: pow*. https://docs.python.org/3/library/functions.html#pow